

Database Design

- Database choice
 - **SQL (PostgreSQL):** users, subscriptions, payments. Needs ACID transactions.
 - **Cassandra:** watch history and progress. Very high write rate, no single point of failure.
 - **MongoDB:** content metadata. Schema differs between movies, series and documentaries.
 - **Elasticsearch:** search index for fuzzy matching and autocomplete.
 - **Redis:** sessions, hot metadata, stream counters.
 - **Object storage:** video files and subtitles, also the CDN origin.
- Entities
 - USER - user_id (PK), email, password_hash, phone, name, country_code, created_at
 - PROFILE - profile_id (PK), user_id (FK), name, is_kids, language
 - SUBSCRIPTION - subscription_id (PK), user_id (FK), plan, start_date, end_date, status
 - PAYMENT - payment_id (PK), subscription_id (FK), amount, txn_reference, status, paid_at
 - CONTENT - content_id (PK), title, type, genre, release_year, duration, language, regions
 - WATCH_HISTORY - history_id (PK), profile_id (FK), content_id (FK), progress_sec, last_watched_at
 - MY_LIST - profile_id (FK), content_id (FK), added_at
- Relationships
 - User to Profile: 1 to N
 - User to Subscription: 1 to 1 active
 - Subscription to Payment: 1 to N
 - Profile to Watch History: 1 to N
 - Profile to Content: N to N, through Watch History and My List
- Design notes
 - Watch history is partitioned by profile_id so continue watching is a single partition read.
 - txn_reference is unique, so a retried payment callback cannot charge twice.
 - SQL tables are sharded by user_id, keeping one user data on one shard.

The ER diagram is on page 2 of the diagram file.